# CS683 Programming Assignment 2

Stride Pattern Prefetching and Instruction-Conscious L2 Replacement

## Setup and metrics

All results come from the provided ChampSim with a single out-of-order core and the hashed-perceptron branch predictor. The cache hierarchy is a 48 KB 12-way L1D (16 MSHRs), a 32 KB L1I, a 512 KB 8-way L2 (1024 sets) and a 2 MB 16-way LLC. Each trace is run for 25M warm-up and 25M simulated instructions, as required. The **baseline** has no prefetcher at any level and LRU everywhere; every speedup is relative to it. Everything was built with `g++-7` on Linux and re-run from scratch for this report; the logs are in `results/raw_logs/`.

- Speedup = $\text{IPC}/\text{IPC}_{\text{baseline}}$. MPKI and prefetch accuracy are taken from the ChampSim statistics (`L1D USEFUL LOAD PREFETCHES ... ACCURACY`).
- L1D coverage = (L1D load $\text{misses}_{\text{baseline}}$ − L1D load $\text{misses}_{\text{pref}}$)/ L1D load $\text{misses}_{\text{baseline}}$.
- "Mean" rows and bars: geometric mean for speedup, arithmetic mean for everything else. Bold marks the best value of each group in the tables.

Everything can be regenerated with `python3 run_all_experiments.py` (builds the binaries and runs all 30 simulations) followed by `python3 plot_results.py`.

# 1 Task 1: Stride Pattern Prefetcher

## 1.1 Task 1A: the IP-stride prefetcher

`ip_stride.l1d_pref` keeps 24 trackers, each holding an IP, the last cache-line address that IP touched, and the last stride. The IP is cut to its low 8 bits, so different loads can alias onto one tracker, and trackers are replaced with true LRU. On every L1D access the stride to the previous line of the same IP is computed. If it equals the stored stride (the same stride seen twice in a row), lines $+1 \cdot s$, $+2 \cdot s$ and $+3 \cdot s$ are prefetched (fixed degree 3), stopping at the 4 KB page boundary. Prefetches fill the L1D while its MSHR is less than half full and only the L2 otherwise, so that prefetches never take MSHRs away from demand misses.

Its limits are what SPP is meant to fix: it only sees constant strides, a single mismatching stride resets training, and the degree is fixed regardless of how useful the prefetches turn out to be.

## 1.2 Task 1B: design

**IP table (64 entries, LRU)**
One entry per load IP with a 16-bit partial IP tag, the last line address, the last stride, a 12-bit stride signature, a 3-bit uniform-confidence counter and a mode bit.

**Dual classification**
When the IP repeats its stride the confidence counter goes up, otherwise it goes down. With confidence $\geq 2$ the IP is in *uniform* mode and is prefetched like IP-stride (line $+ d \cdot s$ for $d = 1..$degree). When the confidence falls below 2 the IP drops to *non-uniform* mode. Uniform is thus preferred whenever the IP is confident, and non-uniform is the fallback, as required.

**Signature and pattern table**
Every stride is folded into the IP's signature, $sig = ((sig \ll 3) \oplus h(s))$ & `0xFFF`, where $h$ packs the stride into 7 bits (6-bit magnitude and a sign bit). The 12-bit signature therefore remembers about the last four strides. The low 9 bits index a 512-entry pattern table (PT) whose entries hold the next stride and a 3-bit confidence. On each access the PT entry of the *previous* signature is trained with the stride just observed; the stored stride is only replaced once its confidence has drained to zero, so one irregular stride does not erase a learned pattern.

**Lookahead**
In non-uniform mode the PT is walked: the predicted stride is added to the address and also folded into a

speculative signature to look up the stride after it. For the sequence $(+1, +2, +2, +2, +3)^n$ this produces several steps of the pattern per trigger. The walk stops at the first entry with confidence below 2.

**Accuracy throttling**

A 256-entry direct-mapped filter records the line of every issued prefetch. A demand load that finds its line there counts as a useful prefetch. Every 512 issued prefetches the epoch accuracy picks the degree (Section 1.4), and both epoch counters are then halved so the next decision still partly remembers the previous epoch.

**Page boundary and fill level**

As in IP-stride, a prefetch that would leave the 4 KB page of the trigger ends the walk, and the MSHR-occupancy rule decides between L1 and L2 fill.

## 1.3 Storage overhead

The simulator keeps some fields in wider C types; the table lists the bits a hardware implementation needs.

| Structure | Fields per entry (bits) | Bits/entry | Entries | Total bits |
|---|---|---|---|---|
| IP table | IP tag 16, last line address 28, last stride 8, signature 12, uniform confidence 3, mode 1, LRU 6, valid 1 | 75 | 64 | 4 800 |
| Pattern table | stride 8, confidence 3 | 11 | 512 | 5 632 |
| Prefetch filter | line tag 16, useful 1, valid 1 | 18 | 256 | 4 608 |
| Throttling state | epoch issued 10, epoch useful 10, degree 2 | 22 | 1 | 22 |
| **Total** | | | | **15 062 bits ≈ 1.84 KB** |

**Table 1:** SPP storage. The stride is limited to ±127 lines (larger jumps restart training), so 8 bits suffice. 28 bits of line address are far more than needed to compute such strides. The filter is indexed by the low 8 bits of the line address and stores the next 16 bits as a partial tag. The epoch counters never exceed 515 (512 plus at most one walk of degree 4), so 10 bits are enough.

## 1.4 Throttling thresholds and degrees

| Measured accuracy | $\geq 70\%$ | 45–70% | 25–45% | $< 25\%$ |
|---|---|---|---|---|
| Degree | 4 | 3 | 2 | 1 |

**How accuracy is measured.** Accuracy is the fraction of issued prefetches that a later demand load hit in the filter, measured over epochs of 512 issued prefetches. 512 is long enough that a few hundred samples give a stable ratio, and short enough to react within a phase (SPP issues hundreds of thousands of prefetches per run). Halving the counters after each decision makes the measurement a moving average, so one unusual epoch cannot flip the degree back and forth.

**Why these thresholds.** We fixed the bands from the costs on both sides, then checked them against alternatives (Table 2).

- *Below 25%* three out of four prefetches are wasted. Each one costs an L2/LLC lookup, possibly a DRAM transfer, and an L1D or L2 slot that a useful line could have used, so we drop to the minimum. The minimum is degree 1 rather than 0: the prefetcher must keep issuing a few prefetches to measure accuracy, otherwise it could never notice that a new phase is predictable.
- *25–45%* is still mostly waste, so we allow one extra line (degree 2).
- *45–70%* is about what the IP-stride baseline achieves on Traces 1 and 2 (38–53%, Table 3). Here SPP uses degree 3, the same as IP-stride's fixed degree, so at comparable accuracy it is as aggressive as the baseline and no more.
- *Above 70%* most prefetches are used and running further ahead hides more latency, so we allow degree 4.

**Why at most 4.** A deeper degree buys little here. Lines are 64 B and a page holds 64 lines, so with strides of several lines a long walk soon hits the page boundary. In non-uniform mode each lookahead step is a guess built on the previous guess, so accuracy falls with depth. Finally, the L1D has only 16 MSHRs: several active IPs at degree $> 4$ would fill half of them quickly, and from then on our prefetches go to the L2 only.

| | Speedup T1 | Speedup T2 | Speedup T3 | Speedup mean | Acc. mean | Cov. mean |
|---|---|---|---|---|---|---|
| Chosen bands 70/45/25 % | 1.0091 | 1.0029 | 1.0183 | 1.0101 | 45.9 | 9.0 |
| Stricter bands 85/65/45 % | 1.0084 | 1.0030 | 1.0158 | 1.0091 | 48.1 | 7.9 |
| Looser bands 55/30/15 % | 1.0088 | 1.0030 | 1.0196 | 1.0105 | 43.8 | 9.6 |
| Fixed degree 1 (no throttling) | 1.0081 | 1.0026 | 1.0134 | 1.0080 | 49.7 | 6.9 |
| Fixed degree 4 (no throttling) | 1.0080 | 1.0027 | 1.0216 | 1.0107 | 39.6 | 10.5 |

**Table 2:** Throttling alternatives (speedup over the no-prefetch baseline, accuracy and coverage are means over the three traces). Logs: `results/exploration/`.

The check confirms the trade-off the bands are meant to strike. Running at degree 4 all the time gives the highest mean speedup (1.0107 vs. 1.0101), but the whole advantage comes from Trace 3, whose patterns are very regular. It is the *worst* of the five settings on Trace 1 and has the lowest accuracy (39.6%, i.e. 60% of its prefetches are wasted). Degree 1 all the time is the most accurate but gives up a fifth of the gain. The chosen bands are the best setting on Trace 1, are never the worst on any trace, and reach 94% of the always-degree-4 gain with 6 points higher accuracy. Looser bands (55/30/15) are within 0.04% of ours and stricter bands are worse on Traces 1 and 3, so the result is not sensitive to the exact boundaries. We kept 70/45/25, which buys robustness on irregular traces for a very small cost on the regular one.

## 1.5 Results

| Trace | Configuration | IPC | Speedup | L1D MPKI | Acc. (%) | Cov. (%) |
|---|---|---|---|---|---|---|
| Trace 1 | Baseline | 0.2323 | 1.0000 | 10.68 | – | – |
| | IP-stride | 0.2335 | 1.0048 | 10.12 | 52.7 | 5.2 |
| | SPP | **0.2345** | **1.0091** | **9.63** | **54.3** | **9.8** |
| Trace 2 | Baseline | 0.2512 | 1.0000 | 13.54 | – | – |
| | IP-stride | 0.2513 | 1.0004 | 13.44 | 38.2 | 0.7 |
| | SPP | **0.2519** | **1.0029** | **13.28** | **40.9** | **1.9** |
| Trace 3 | Baseline | 0.3821 | 1.0000 | 3.40 | – | – |
| | IP-stride | 0.3829 | 1.0021 | 3.32 | 16.0 | 2.5 |
| | SPP | **0.3891** | **1.0183** | **2.89** | **42.5** | **15.2** |
| Mean† | Baseline | 0.2885 | 1.0000 | 9.21 | – | – |
| | IP-stride | 0.2892 | 1.0024 | 8.96 | 35.6 | 2.8 |
| | SPP | **0.2918** | **1.0101** | **8.60** | **45.9** | **9.0** |

**Table 3:** Task 1 results. †geometric mean for speedup, arithmetic mean otherwise.

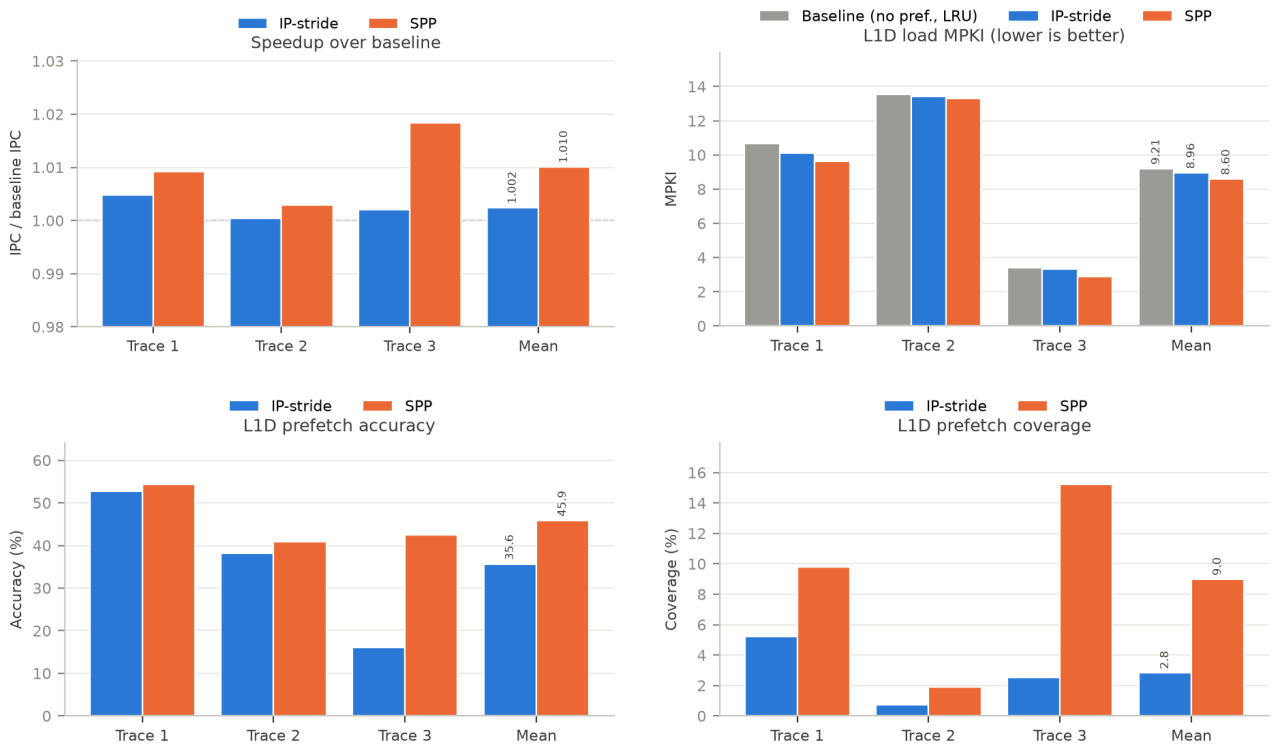



**Figure 1:** Task 1: speedup, L1D load MPKI, prefetch accuracy and coverage.

SPP beats IP-stride on every trace and every metric. On average it lowers L1D load MPKI from 9.21 to 8.60 (IP-stride: 8.96), with 45.9% accuracy and 9.0% coverage (IP-stride: 35.6% and 2.8%), for a mean speedup of 1.0101 against 1.0024.

- **Trace 3** gains most: coverage rises from 2.5% to 15.2% and accuracy from 16.0% to 42.5%. IP-stride is both inaccurate and rarely triggered here, which points to access patterns it cannot follow: strides that are not constant, or loads aliasing on its 8-bit IP tag. Both are handled by SPP's signature path and 16-bit tag.
- **Trace 1** has the most regular strides: IP-stride already reaches 52.7% accuracy, and SPP nearly doubles its coverage (5.2% → 9.8%) through deeper prefetching at high accuracy.
- **Trace 2** gains least (coverage 1.9%). It is dominated by instruction misses: its L2 instruction MPKI (18.85) is more than twice its data MPKI (8.50), and an L1D data prefetcher cannot help those. This is the trace where Task 2 matters.

The absolute speedups are small (1–2%) because these traces are bound by instruction misses and by load misses with no stride structure. Coverage, not accuracy, limits SPP.

# 2 Task 2A: Instruction-conscious L2 replacement (I-RRIP)

## 2.1 Interface change

`update_replacement_state` gained a parameter `isInstruction`. Every call site in `cache.cc` passes the packet's `instruction` flag. On a hit the flag of the resident block is used if it is already set, so a line that has ever been fetched as code stays an instruction line. The BTB calls in `ooo_cpu.cc` pass 1. All other replacement policies take the parameter with default 0 and ignore it.

## 2.2 Policy and rationale

I-RRIP is SRRIP with a 2-bit RRPV (0 = re-referenced soon, 3 = evict next) plus one bit per line saying whether the line holds code. All three parts of the policy are biased towards instruction lines:

**Insertion: instruction lines at RRPV 1, data at RRPV 2**
Data keeps the normal SRRIP insertion point, so data behaves exactly as under SRRIP. Instruction lines start one step closer to the MRU end and therefore survive one more aging round before they become victims. We

did not insert code at RRPV 0. That would make a freshly fetched line look as if it had already been reused, and lines executed only once (initialisation code, cold paths) would then crowd out data that really is reused.

**Promotion: instruction hits to RRPV 0, data hits to RRPV 1**

A hit on code is strong evidence of a loop or a hot function, so it gets the strongest protection. A data hit is still promoted, but only to RRPV 1. Hot data therefore stays well protected, while a reused instruction line still outranks it when the set is under pressure.

**Eviction: data first among lines at RRPV 3**

The victim search first looks for a *data* line at RRPV 3 and takes an instruction line only if none is found. Only then are all RRPVs incremented, as in SRRIP. An instruction line is therefore evicted only when it is as old as the oldest data in the set. Data is never protected absolutely, so code cannot occupy a set indefinitely.

**Why bias towards code at all?** An L2 instruction miss stalls fetch: the frontend has nothing to decode, and the whole out-of-order window drains. Many L2 data misses, in contrast, overlap with independent work in the ROB. A missing instruction line therefore costs more cycles than a missing data line, and the traces here have an L2 instruction MPKI as large as or larger than the data MPKI (Table 4). **Why this mix of mild biases rather than one strong rule?** Each of the three changes is one RRPV step. Together they make an instruction line live roughly one aging round longer than a comparable data line without ever pinning it. A stronger rule, such as never evicting code while data is present, would protect a large code footprint perfectly but starve data in phases where code is cold, which is exactly the problem Task 2B addresses.

## 2.3 Results

| Trace | Configuration | IPC | Speedup | L2C MPKI | L2 Instr. MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|
| Trace 1 | Baseline | **0.2323** | **1.0000** | 17.70 | 10.34 | **7.36** |
| | I-RRIP | 0.2322 | 0.9994 | **17.62** | **9.89** | 7.72 |
| Trace 2 | Baseline | 0.2512 | 1.0000 | 27.36 | 18.85 | **8.50** |
| | I-RRIP | **0.2566** | **1.0217** | **22.23** | **13.05** | 9.18 |
| Trace 3 | Baseline | **0.3821** | **1.0000** | 3.95 | 1.12 | **2.83** |
| | I-RRIP | 0.3820 | 0.9998 | **3.90** | **1.02** | 2.88 |
| Mean† | Baseline | 0.2885 | 1.0000 | 16.34 | 10.11 | **6.23** |
| | I-RRIP | **0.2903** | **1.0069** | **14.58** | **7.99** | 6.59 |

**Table 4:** Task 2A results (L2 MPKI = L2C load MPKI). †geometric mean for speedup.

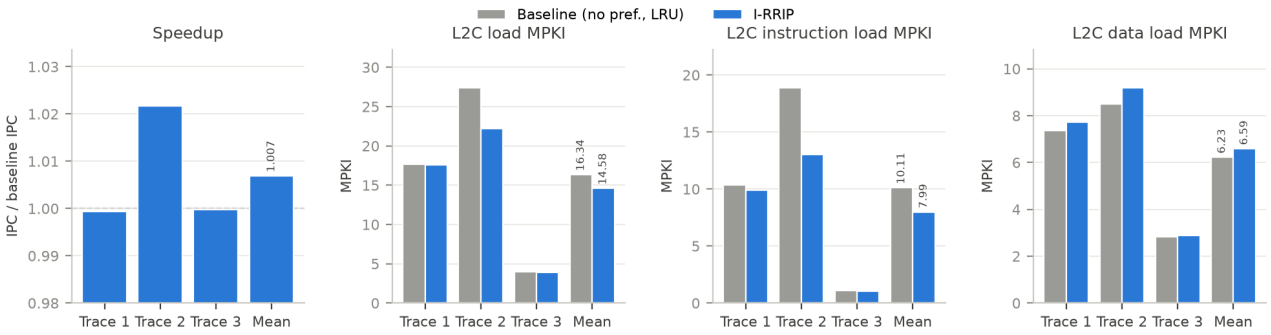



**Figure 2:** Task 2A: speedup and L2C load / instruction / data MPKI, LRU baseline vs. I-RRIP.

I-RRIP does what it is designed for: mean L2 instruction MPKI falls from 10.11 to 7.99 (−21%), and on Trace 2 from 18.85 to 13.05 (−31%). The price is a smaller rise in data MPKI (6.23 → 6.59), so total L2 MPKI still drops from 16.34 to 14.58.

- **Trace 2** (large code footprint) gains 1.0217×, which shows how costly L2 instruction misses are.
- **Trace 1**: instruction MPKI falls by 0.45 and data MPKI rises by 0.36. On this trace the two roughly cancel, and IPC changes by −0.06%.
- **Trace 3**: the code footprint fits in L2 (instruction MPKI 1.12), so there is little to protect and IPC is unchanged (−0.02%).

Note that the baseline here is LRU, not SRRIP, so these numbers include the difference between the two base policies. The Task 3 diagnostics show that on Trace 1 SRRIP alone is already slightly behind LRU.

# 3 Task 2B: Dynamic I-RRIP

## 3.1 Design

**Leader sets**

32 of the 1024 L2 sets (sets $0, 32, 64, \dots$) are leaders. They always run plain SRRIP and record, for every access, whether it was an instruction access and whether it missed.

**Decision**

The follower sets (all others) run I-RRIP when

$$M = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{all accesses} + \text{all misses}} \geq T,$$

and SRRIP otherwise. The decision is re-evaluated on every access, so followers switch as soon as the leaders' statistics cross $T$.

**Phase tracking**

When the total number of counted events reaches 16 384, all four counters are halved, giving an exponentially decaying window. $M$ therefore follows the current program phase instead of the whole run. Until 64 events have been seen, followers default to I-RRIP.

**Why the leaders run SRRIP.** The leaders measure the workload, not the policy. If they ran I-RRIP, protecting code would lower the instruction miss count and push $M$ down, which would switch the followers back to SRRIP and in turn raise the code misses again, so the policy would oscillate. Leaders with an unbiased policy see the instruction pressure that I-RRIP is meant to relieve.

**Why this formula.** Adding misses to accesses weights the metric towards traffic that actually hurts: an instruction stream that mostly hits contributes accesses only, while one that misses contributes twice. A large $M$ means code is a big share of L2 pressure, which is when protecting it pays off.

**Why 32 leader sets.** Leaders give up the I-RRIP benefit, so they should be few. They must still see enough traffic to estimate $M$ reliably and to follow phase changes quickly. 32 sets are 3.1% of the cache (256 of 8192 lines) and receive about 1 in 32 L2 accesses. Because the counters are halved at 16K leader events, the estimate is always based on 8K–16K samples. For a ratio around 0.3 that gives a standard error of about 0.005, far below the gaps between the thresholds we study. At the same time, one window covers only about 250K–500K L2 accesses, a few million instructions, so phase changes are noticed quickly. With 16 leaders the window would be twice as long; with 64, twice as many sets would give up I-RRIP for no gain in accuracy. 32 leaders per 1024 sets is also the size used in the original set-dueling proposal (DIP, Qureshi et al., ISCA 2007). Placing them every 32 sets spreads them evenly over the index space, so no single hot region dominates the sample.

## 3.2 Sensitivity study: decision threshold

| | Speedup | | | L2C MPKI | | | L2 Instr. MPKI | | | L2 Data MPKI | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Threshold | T1 | T2 | T3 | T1 | T2 | T3 | T1 | T2 | T3 | T1 | T2 | T3 |
| 0.15 | 0.9993 | 1.0207 | 0.9998 | 17.63 | 22.36 | 3.90 | 9.92 | 13.23 | 1.03 | 7.71 | 9.13 | 2.87 |
| 0.20 | 0.9997 | 1.0207 | 0.9998 | 17.61 | 22.35 | 3.90 | 9.90 | 13.23 | 1.03 | 7.70 | 9.13 | 2.87 |
| 0.25 | 0.9996 | 1.0198 | 0.9994 | 17.62 | 22.55 | 3.95 | 9.92 | 13.43 | 1.08 | 7.70 | 9.12 | 2.88 |
| 0.30 | 0.9997 | 1.0192 | 0.9994 | 17.62 | 22.69 | 3.98 | 9.92 | 13.59 | 1.11 | 7.70 | 9.10 | 2.87 |
| 0.35 | 0.9998 | 1.0185 | 0.9993 | 17.62 | 22.83 | 3.99 | 9.93 | 13.75 | 1.12 | 7.69 | 9.08 | 2.87 |

**Table 5:** Dynamic I-RRIP for five thresholds $T$ (T1–T3 = Trace 1–3).

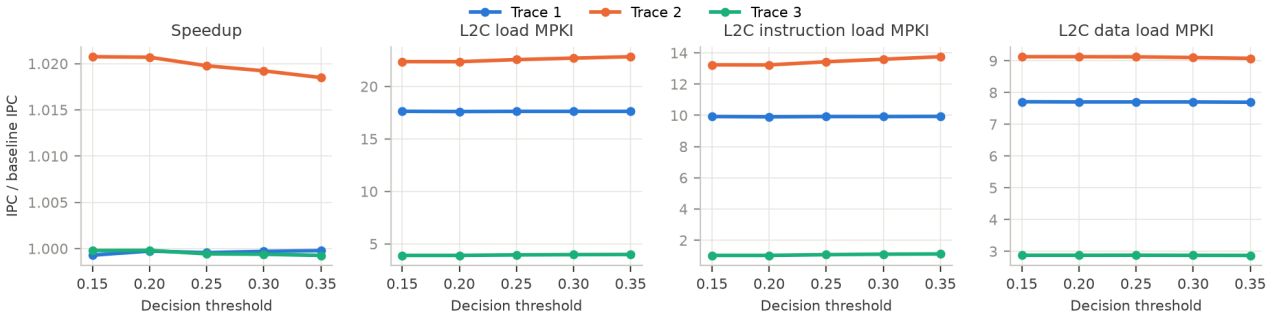



**Figure 3:** Task 2B sensitivity: speedup and L2 MPKI versus the decision threshold.

**Observations.** Lower thresholds are better on these traces. On Trace 2, speedup rises steadily from 1.0185 at $T = 0.35$ to 1.0207 at $T = 0.15$, while L2 instruction MPKI falls from 13.75 to 13.23. A lower $T$ keeps the followers in I-RRIP mode for a larger part of the run, and on a trace with a big code footprint that is what pays off. Trace 3 shows the same direction on a much smaller scale. Trace 1 is flat (all five within 0.05%), because there the extra instruction hits and the lost data hits cancel out, as in Task 2A. Data MPKI barely moves with $T$ on any trace: in the phases where the leaders see little code, switching to SRRIP costs little.

**Chosen threshold:** $T = 0.15$**.** In this study 0.15 and 0.20 are tied (identical on Traces 2 and 3; 0.20 is 0.04% ahead on Trace 1). With SPP enabled (Task 3), 0.15 was better than 0.25 on every trace (Table 8), so we use $T = 0.15$ for both tasks.

## 3.3 Results

| Trace | Configuration | IPC | Speedup | L2C MPKI | L2 Instr. MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|
| Trace 1 | Baseline | **0.2323** | **1.0000** | 17.70 | 10.34 | **7.36** |
| | I-RRIP | 0.2322 | 0.9994 | **17.62** | **9.89** | 7.72 |
| | Dyn. I-RRIP | 0.2322 | 0.9993 | 17.63 | 9.92 | 7.71 |
| Trace 2 | Baseline | 0.2512 | 1.0000 | 27.36 | 18.85 | **8.50** |
| | I-RRIP | **0.2566** | **1.0217** | **22.23** | **13.05** | 9.18 |
| | Dyn. I-RRIP | 0.2564 | 1.0207 | 22.36 | 13.23 | 9.13 |
| Trace 3 | Baseline | **0.3821** | **1.0000** | 3.95 | 1.12 | **2.83** |
| | I-RRIP | 0.3820 | 0.9998 | **3.90** | **1.02** | 2.88 |
| | Dyn. I-RRIP | 0.3820 | 0.9998 | **3.90** | 1.03 | 2.87 |
| Mean† | Baseline | 0.2885 | 1.0000 | 16.34 | 10.11 | **6.23** |
| | I-RRIP | **0.2903** | **1.0069** | **14.58** | **7.99** | 6.59 |
| | Dyn. I-RRIP | 0.2902 | 1.0066 | 14.63 | 8.06 | 6.57 |

**Table 6:** Task 2B results with the chosen threshold, next to static I-RRIP. †geometric mean for speedup.

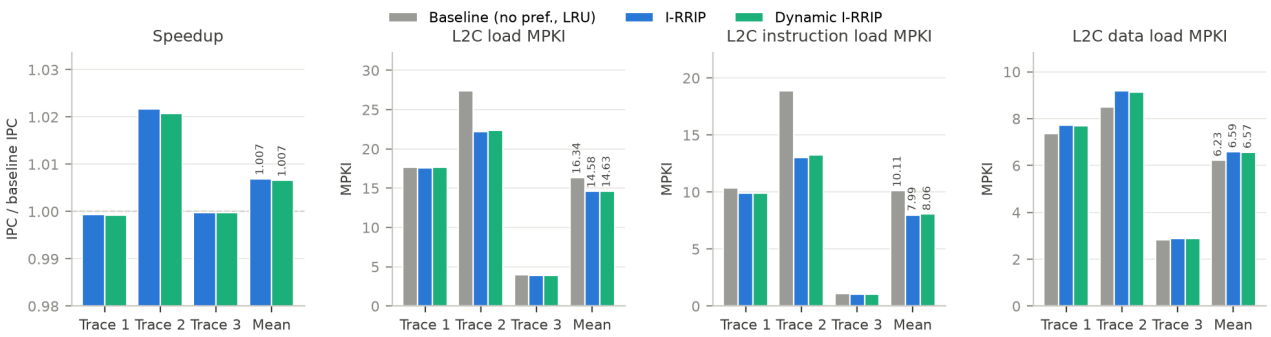



**Figure 4:** Task 2B: LRU baseline, static I-RRIP and dynamic I-RRIP.

Dynamic I-RRIP reaches a mean speedup of 1.0066, and it is close to static I-RRIP (1.0069) on every trace. Because all three traces have a substantial share of instruction traffic at L2, the switch rarely finds a phase where

SRRIP is better, so it cannot gain much over always prioritising code. Its small cost on Trace 2 (instruction MPKI 13.23 vs. 13.05) is the price of the 32 leader sets, which always run SRRIP. The value of the dynamic version is robustness: on a workload whose code fits in L1I/L2, the followers fall back to SRRIP instead of giving data lines a needless handicap. These three traces do not contain such a phase.

# 4 Task 3: SPP with prefetch-aware dynamic I-RRIP

## 4.1 How the replacement policy handles prefetched lines

With SPP enabled, the L2 receives a third kind of fill besides demand instruction and demand data lines: lines requested by the L1D prefetcher (packet type `PREFETCH`). Every SPP prefetch that misses in the L2 is filled there, including the ones that also go to the L1D. `dyn_irrip.l2c_repl` keeps one extra bit per line, `l2_is_pref`, and treats these lines as follows:

**Insertion at RRPV 3, marked as prefetch**
A prefetched line has not been referenced by the program yet. It is only a prediction, and SPP's accuracy is about 40–55% (Table 7), so roughly half of these lines will never be used. Inserting them at RRPV 3 makes them the next victim in their set, so a wrong prediction can displace at most another dead line, never a protected instruction line or reused data. Prefetches issued while the L1D MSHR is less than half full are filled into the L1D as well and are consumed there, so their L2 copy rarely needs to survive long.

**Evicted first while unused**
At RRPV 3 the victim search first takes an *unused* prefetched line, then data, then code. This applies in both I-RRIP and SRRIP mode, so the instruction protection of Task 2 is never paid for by SPP's mistakes.

**Demand hit clears the mark**
When a demand access hits a prefetched line, the prediction was right. The mark is cleared and the line is promoted like any other hit (RRPV 0 for code, 1 for data in I-RRIP mode, 0 in SRRIP mode). From then on it competes as a normal line.

The point of these rules is to keep the two mechanisms from fighting each other: prefetched lines should never age the instruction lines that I-RRIP is trying to protect. The measured effect of each rule is discussed with the design variants below.

## 4.2 Results

| Trace | Configuration | IPC | Speedup | Acc. (%) | Cov. (%) | L1D MPKI | L2C MPKI | L2 Instr. MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|---|---|---|
| Trace 1 | Baseline | 0.2323 | 1.0000 | – | – | 10.68 | 17.70 | 10.34 | 7.36 |
| | SPP | **0.2345** | **1.0091** | **54.3** | **9.8** | **9.63** | 16.83 | 10.37 | **6.46** |
| | Dyn. I-RRIP | 0.2322 | 0.9993 | – | – | 10.68 | 17.63 | 9.92 | 7.71 |
| | SPP + Dyn. I-RRIP | 0.2343 | 1.0085 | **54.3** | **9.8** | **9.63** | **16.68** | **9.91** | 6.77 |
| Trace 2 | Baseline | 0.2512 | 1.0000 | – | – | 13.54 | 27.36 | 18.85 | 8.50 |
| | SPP | 0.2519 | 1.0029 | **40.9** | **1.9** | **13.28** | 27.30 | 19.02 | **8.29** |
| | Dyn. I-RRIP | 0.2564 | 1.0207 | – | – | 13.54 | 22.36 | 13.23 | 9.13 |
| | SPP + Dyn. I-RRIP | **0.2574** | **1.0247** | 40.8 | **1.9** | **13.28** | **22.08** | **13.19** | 8.89 |
| Trace 3 | Baseline | 0.3821 | 1.0000 | – | – | 3.40 | 3.95 | 1.12 | 2.83 |
| | SPP | **0.3891** | 1.0183 | **42.5** | **15.2** | **2.89** | 3.30 | 1.15 | **2.15** |
| | Dyn. I-RRIP | 0.3820 | 0.9998 | – | – | 3.40 | 3.90 | **1.03** | 2.87 |
| | SPP + Dyn. I-RRIP | **0.3891** | **1.0184** | 42.4 | **15.2** | **2.89** | **3.21** | **1.03** | 2.19 |
| Mean† | Baseline | 0.2885 | 1.0000 | – | – | 9.21 | 16.34 | 10.11 | 6.23 |
| | SPP | 0.2918 | 1.0101 | **45.9** | **9.0** | **8.60** | 15.81 | 10.18 | **5.63** |
| | Dyn. I-RRIP | 0.2902 | 1.0066 | – | – | 9.21 | 14.63 | 8.06 | 6.57 |
| | SPP + Dyn. I-RRIP | **0.2936** | **1.0172** | 45.8 | **9.0** | **8.60** | **13.99** | **8.04** | 5.95 |

**Table 7:** Task 3 results (dynamic I-RRIP at $T = 0.15$). Acc./Cov. are L1D prefetch accuracy and coverage. †geometric mean for speedup, arithmetic mean otherwise.

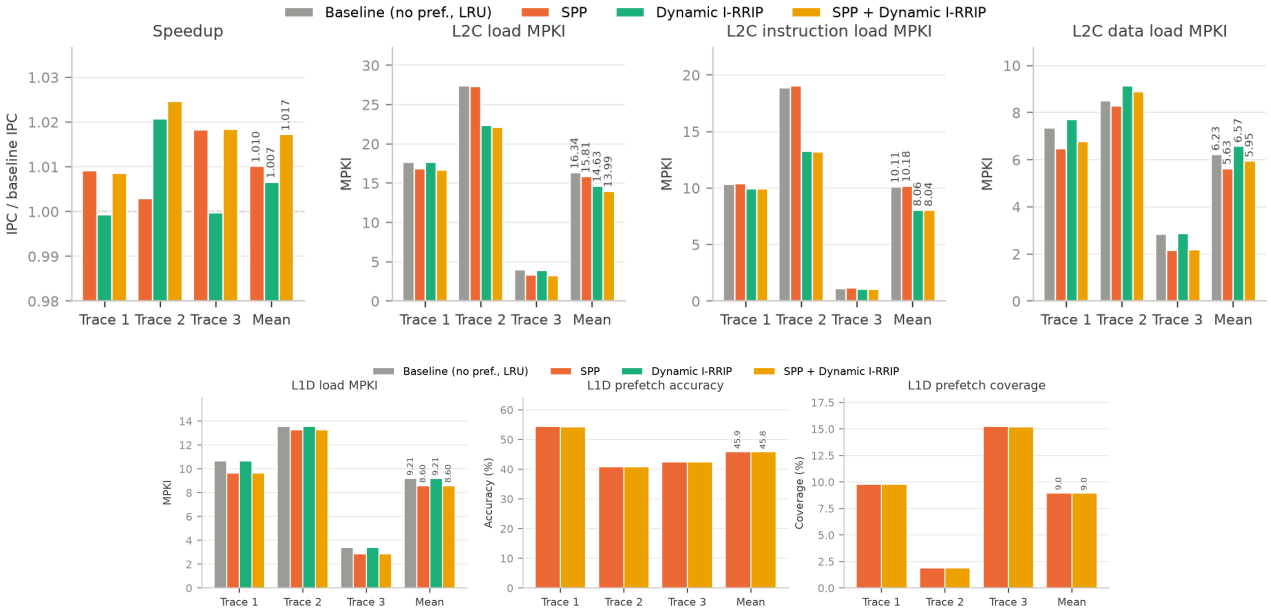



**Figure 5:** Task 3: speedup and L2 MPKI (top); L1D MPKI, prefetch accuracy and coverage (bottom).

The combination is the best configuration on average: mean speedup 1.0172, against 1.0101 for SPP alone and 1.0066 for dynamic I-RRIP alone. That is slightly more than the two gains multiplied ($1.0101 \times 1.0066 \approx 1.0168$), so the two mechanisms do not get in each other's way. SPP's accuracy and coverage are unchanged by the L2 policy, while the L2 now has both fewer instruction misses (10.18 → 8.04) and fewer data misses than dynamic I-RRIP alone (6.57 → 5.95).

Per trace, the combination beats both components on Traces 2 and 3, but not on Trace 1:

- **Trace 2**: 1.0247 vs. 1.0029 (SPP) and 1.0207 (dyn. I-RRIP), clearly ahead of both.
- **Trace 3**: 1.0184 vs. 1.0183: ahead, but only by 0.01%. The code fits in L2, so there is almost nothing for the replacement policy to add.
- **Trace 1**: 1.0085 vs. 1.0091, i.e. 0.06% *behind* SPP alone (still well ahead of dynamic I-RRIP alone).

## 4.3 Design exploration and the Trace 1 gap

To find the cause of the Trace 1 gap and to measure what the prefetch handling contributes, we ran the variants in Table 8. Each is a small change to `dyn_irrip.l2c_repl`; the exact diffs and logs are in `results/exploration/`.

| | Speedup T1 | Speedup T2 | Speedup T3 | Speedup mean | L2I T1 | L2D T1 |
|---|---|---|---|---|---|---|
| SPP alone (LRU at L2) | 1.0091 | 1.0029 | 1.0183 | 1.0101 | 10.37 | 6.46 |
| **Submitted**: $T = 0.15$ | 1.0085 | 1.0247 | 1.0184 | 1.0172 | 9.91 | 6.77 |
| Submitted policy, $T = 0.25$ | 1.0083 | 1.0237 | 1.0178 | 1.0166 | 9.90 | 6.77 |
| Submitted policy, $T = 0.35$ | – | – | 1.0178 | – | – | – |
| (N) no prefetch awareness, $T = 0.15$ | 1.0083 | – | – | – | 9.98 | 6.78 |
| (Q) prefetches inserted at RRPV 1, $T = 0.15$ | 1.0085 | – | – | – | 9.97 | 6.77 |
| (V1) prefetches inserted at RRPV 2, $T = 0.25$ | 1.0083 | 1.0235 | 1.0179 | 1.0165 | 9.96 | 6.75 |
| (V2) no promotion on prefetch-request hit, $T = 0.25$ | – | – | 1.0178 | – | – | – |
| (E) data hits to RRPV 0, $T = 0.15$ | 1.0084 | 1.0212 | 1.0182 | 1.0159 | 10.07 | 6.60 |
| (H) instructions inserted at RRPV 0, $T = 0.15$ | 1.0081 | 1.0237 | 1.0184 | 1.0167 | 9.87 | 6.97 |
| Followers forced to SRRIP | 1.0086 | – | – | – | 10.56 | 6.20 |
| Followers forced to I-RRIP | 1.0086 | – | – | – | 9.91 | 6.77 |

**Table 8:** SPP + dynamic I-RRIP variants ("–": not run on that trace). L2I/L2D T1: L2 instruction / data load MPKI on Trace 1.

- **The gap comes from the RRIP base policy, not from the combination.** With the followers forced to plain SRRIP, Trace 1 reaches the same speedup as with them forced to I-RRIP, and both are about 0.05% behind LRU. SRRIP keeps more data (L2 data MPKI 6.20 vs. 6.46 under LRU) but loses more code (10.56 vs. 10.37); I-RRIP does the opposite (6.77 / 9.91). Neither trade pays on Trace 1, which is the same picture as in Task 2A. The assignment fixes RRIP as the base policy, so the remaining choice is between the two

RRIP modes, and they are equal here.

- **Prefetch handling matters little on these traces.** Dropping prefetch awareness (N), protecting prefetched lines (Q, RRPV 1) or inserting them at RRPV 2 (V1) all change speedup by at most 0.02%, in both directions depending on the trace. Prefetch fills are only 9%, 3% and 21% of all L2 fills on Traces 1, 2 and 3, and many of them are filled into the L1D as well, so the L2 policy's treatment of them has little room to matter. None of the alternatives is consistently better. We keep the evict-first rule because it is the safe default: it can only discard lines that were never used, and it guarantees that a phase of inaccurate prefetching cannot evict protected code.
- **The threshold matters more than the prefetch rules.** At $T = 0.15$ the combination is ahead of the $T = 0.25$ version on all three traces, the same trend as in Task 2B.
- Making the instruction bias stronger (H) or weaker (E) did not help either.

# Summary

| Configuration | Trace 1 | Trace 2 | Trace 3 | Geomean |
|---|---|---|---|---|
| IP-stride | 1.0048 | 1.0004 | 1.0021 | 1.0024 |
| SPP (Task 1) | 1.0091 | 1.0029 | 1.0183 | 1.0101 |
| I-RRIP (Task 2A) | 0.9994 | 1.0217 | 0.9998 | 1.0069 |
| Dynamic I-RRIP (Task 2B) | 0.9993 | 1.0207 | 0.9998 | 1.0066 |
| SPP + Dyn. I-RRIP (Task 3) | 1.0085 | 1.0247 | 1.0184 | **1.0172** |

**Table 9:** Speedup over the no-prefetch, LRU baseline.

SPP raises L1D coverage and accuracy over IP-stride on every trace for 1.84 KB of state. I-RRIP cuts L2 instruction misses by a fifth, and by almost a third on the code-heavy Trace 2. Combining SPP with the prefetch-aware dynamic I-RRIP gives the highest mean speedup and beats both components on Traces 2 and 3. On Trace 1 it is 0.06% behind SPP alone, a gap we traced to RRIP-vs.-LRU behaviour on that trace rather than to the interaction between prefetching and replacement.